VLSI Task 4: PWM Generator

Verilog RTL and Testbench — Xilinx Vivado

Design source: **pwm_generator.v**. Simulation source: **tb_pwm_generator.v**.

1. Verilog RTL Code

```
module pwm_generator (
    input clk,
    input rst_n,
    input [7:0] duty_cmd,
    output reg pwm_out
);

    reg [7:0] counter;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            counter <= 8'd0;
            pwm_out <= 1'b0;
        end
        else begin
            counter <= counter + 1'b1;

            if (counter < duty_cmd)
                pwm_out <= 1'b1;
            else
                pwm_out <= 1'b0;
        end
    end

endmodule
```

2. Verilog Testbench Code

```
`timescale 1ns / 1ps

module tb_pwm_generator;

    reg clk = 0;
    reg rst_n = 0;
    reg [7:0] duty_cmd = 0;

    wire pwm_out;

    pwm_generator uut (
        .clk(clk),
        .rst_n(rst_n),
        .duty_cmd(duty_cmd),
        .pwm_out(pwm_out)
    );

    always #5 clk = ~clk;

    initial begin
        #20;
        rst_n = 1;

        // 25% duty cycle
        duty_cmd = 8'd64;
        #2560;

        // 50% duty cycle
        duty_cmd = 8'd128;
        #2560;

        // 75% duty cycle
        duty_cmd = 8'd192;
        #2560;

        // 0% duty cycle
        duty_cmd = 8'd0;
        #2560;

        $finish;
    end

endmodule
```

I/O definition

Inputs: clk (clock), rst_n (active-low reset), duty_cmd[7:0] (duty-cycle command from 0 to 255). Output: pwm_out (PWM signal).

Expected behavior

duty_cmd=64 gives approximately 25% HIGH time, 128 gives approximately 50%, 192 gives approximately 75%, and 0 gives 0%. With this simple counter/comparator design, duty_cmd=255 gives approximately 99.6%, not exactly 100%.